

Experimento de escalabilidad H4.3.1

Reprocesamiento masivo de perfiles con nuevas fuentes Open Data

Resultado principal

Con recursos totales comparables (3 CPU / 3 GiB), la topologia horizontal sostuvo 1.399,98 TPS sin iteraciones descartadas. La vertical sostuvo 1.390,60 TPS y descarto 1.125 iteraciones. Ambas cumplieron p95 <= 400 ms y p99 <= 800 ms.

Documento	Informe de resultados y analisis
Fecha	24 de septiembre de 2026
Commit	1a50feb8d83eeae66b2fd1977974567f9d347bfc
Entorno	Docker Desktop 29.6.2 / macOS (evidencia local)
Estado AWS	Pendiente de acceso a cuenta e infraestructura

1. ASR, hipotesis y criterios

H4.3.1 solicita recalcular al menos 10 millones de perfiles en menos de dos horas al incorporar nuevas fuentes. Esto equivale a 1.388,89 perfiles por segundo. Cada POST /perfilamiento/reprocesar calcula y persiste de forma idempotente un perfil; por tanto, una respuesta exitosa representa un perfil reprocesado.

Hipotesis

Tres instancias de 1 CPU/1 GiB detras de un balanceador mantendran mayor throughput util que una sola instancia vertical de 3 CPU/3 GiB, conservando recursos totales comparables.

Diseno

Variable	Definicion
Carga	200, 500, 800, 1.100 y 1.400 TPS; 2 minutos por nivel
Vertical	1 instancia; 1/1, 2/2 y 3 CPU/GiB
Horizontal	3 instancias de 1 CPU/1 GiB + Nginx least_conn
Base comun	PostgreSQL 16; 2 CPU/2 GiB
Aceptacion	p95 <= 400 ms; p99 <= 800 ms; errores < 1%
Capacidad	Throughput >= 1.388,89 perfiles/s

2. Vista de arquitectura del experimento

Tactica vertical

k6	->	API NestJS 1 instancia CPU/memoria creciente	->	PostgreSQL 16
----	----	--	----	---------------

Tactica horizontal

k6	->	Nginx least_conn	->	API 1 1 CPU/1 GiB	PostgreSQL 16
				API 2 1 CPU/1 GiB	base compartida
				API 3 1 CPU/1 GiB	

Elementos involucrados

Controller y service de perfilamiento, TypeORM, PostgreSQL, generador k6 de tasa de llegada constante, limites de recursos de Docker y balanceador Nginx. Las dos tacticas usan la misma version de aplicacion, base y secuencia de carga.

3. Resultados verticales

Recursos	TPS of.	TPS logr.	p95	p99	Err.%	Desc.
1/1	200	200,01	3,02	<=800*	0	0
1/1	500	500,01	1,06	<=800*	0	0
1/1	800	800,01	0,96	<=800*	0	0
1/1	1.100	1.099,99	1,40	<=800*	0	0
1/1	1.400	1.372,88	5,89	<=800*	0	1.708
2/2	200	200,00	2,32	4,43	0	0
2/2	500	500,00	1,21	207,20	0	0
2/2	800	798,09	1,15	107,83	0	227
2/2	1.100	1.075,39	4,46	636,75	0	2.951
2/2	1.400	1.390,13	1,53	130,42	0	1.083
3/3	200	200,00	2,69	19,94	0	0
3/3	500	500,00	1,01	3,06	0	0
3/3	800	798,64	0,98	9,96	0	163
3/3	1.100	1.099,43	1,30	11,75	0	70
3/3	1.400	1.390,60	1,35	11,19	0	1.125

* El primer resumen aprobo p99 <= 800 ms, pero no conservo el valor numerico. No se inventa el dato faltante.

4. Resultados horizontales y comparacion

Inst.	Rec. c/u	TPS of.	TPS logr.	p95	p99	Err.%	Desc.
3	1/1	200	200,00	2,70	5,09	0	0
3	1/1	500	499,88	2,20	14,90	0	0
3	1/1	800	799,95	1,80	7,43	0	0
3	1/1	1.100	1.089,20	3,27	120,37	0,403	1.294
3	1/1	1.400	1.399,98	2,81	12,27	0,011	0

Comparacion a 1.400 TPS

Topologia	Recursos API	TPS logrados	p95	p99	Descartadas
Vertical	1 x 3 CPU/3 GiB	1.390,60	1,35	11,19	1.125
Horizontal	3 x 1 CPU/1 GiB	1.399,98	2,81	12,27	0

La horizontal absorbio completamente la carga objetivo. La vertical supero la tasa minima derivada del ASR, pero mostro descartes. Todas las latencias permanecieron ampliamente dentro de los limites.

5. Conclusion, decision y limitaciones

Conclusion

La hipotesis queda respaldada en el entorno local. Con recursos totales comparables, varias instancias detras de un balanceador ofrecieron mayor throughput util y cero descartes a 1.400 TPS. Se recomienda escalamiento horizontal para el reprocesamiento masivo y vertical como ajuste inicial o contingencia.

Alcance real de la evidencia

La tasa horizontal observada equivale a aproximadamente 10,08 millones de perfiles en dos horas si se sostiene. Sin embargo, esta corrida duro dos minutos por nivel, no dos horas. Tampoco se midio un canal en linea simultaneo porque el repositorio aun no lo implementa.

Limitaciones y trabajo pendiente

Limitacion	Accion necesaria
Ejecucion local compartida	Repetir con generador separado en AWS
Sin evidencia AWS	Desplegar API en EC2/ECS, RDS y ALB; capturar CloudWatch
Sin canal en linea	Implementar consulta y medirla en paralelo
Duracion corta	Ejecutar 1.400 TPS por 2 horas o procesar 10 millones
Una repeticion	Realizar al menos tres repeticiones por configuracion

6. Reproducibilidad y evidencias

Archivos versionados

Dockerfile; compose.vertical.yml; compose.horizontal.yml; nginx.conf; k6/reprocesamiento.js; run.sh; README.md; RESULTADOS.md. Los resúmenes JSON y logs se conservan localmente bajo experiments/h4.3.1/results/ y se excluyen de Git por su variabilidad.

Comandos

Vertical: API_CPUS=3 API_MEMORY=3g npm run experiment:h431 -- vertical

Horizontal: INSTANCE_CPUS=1 INSTANCE_MEMORY=1g npm run experiment:h431 -- horizontal

Evidencia pendiente para el video

Mostrar el despliegue en consola AWS, recorrido del código, ejecución k6 y resultados/CloudWatch. Este informe no afirma que la medición local sea AWS.